МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»»
Институт компьютерных наук
Кафедра инженерной кибернетики

Курсовая работа
по дисциплине «Методы обработки изображений»

Сегментация КТ-срезов грецкого ореха с использованием сохраняющей границы фильтрации

Выполнил: студент гр. БПМ-22-ФИН-1
Самсонов Н. О.

Проверил: доцент, к.т.н.
Полевой Д. В.

Москва, 2026

Содержание

1 Введение **2**

2 Общая информация **3**

2.1 Постановка задачи 3

2.2 Используемые данные 3

3 Описание алгоритма **4**

3.1 Общая схема 4

3.2 Пошаговое описание 4

3.3 Изменения в версии v2 11

4 Пользовательское описание **13**

4.1 Режим сегментации 13

4.2 Режим оценки качества 13

5 Результаты **15**

5.1 Критерии качества 15

5.2 Средние метрики версии v1 15

5.3 Средние метрики версии v2 15

5.4 Анализ карт ошибок 16

5.5 Проблемные случаи 17

6 Техническое описание **18**

6.1 Установка 18

6.2 Как воспроизвести результаты 18

7 Заключение **20**

1 Введение

Компьютерная томография позволяет исследовать внутреннюю структуру объекта без его разрушения. В данной работе рассматриваются КТ-срезы грецкого ореха. На таких изображениях хорошо видны основные части объекта, но границы между ними выражены не идеально: на срезах присутствуют шум, неравномерность яркости и тонкие внутренние структуры, которые сложно отделить автоматически.

Цель работы — реализовать алгоритм, который по одному КТ-срезу выделяет три класса: внешнюю скорлупу, ядро и внутренние перегородки. Алгоритм должен быть написан на C++ с использованием OpenCV, запускаться как консольное приложение и сохранять результаты так, чтобы их можно было проверить визуально и количественно.

2 Общая информация

2.1 Постановка задачи

На вход подаётся серое изображение КТ-среза грецкого ореха. Требуется автоматически построить карту классов, где каждому пикселю сопоставляется одна из меток:

- 0 — фон;
- 1 — `shell` (скорлупа);
- 2 — `kernel` (ядро);
- 3 — `septa` (внутренние перегородки).

Иными словами, для каждого пикселя нужно определить, к какой части ореха он относится. Качество сегментации оценивается сравнением предсказанной карты классов с ручной эталонной разметкой.

2.2 Используемые данные

В работе используется набор `Walnut2/Reconstructions`. Исходные TIFF-срезы лежат в каталоге `dataset/Walnut2/Reconstructions`. Для количественной оценки качества вручную размечены четыре среза, которые лежат в папке `gt_final`: `193`, `226`, `245`, `248`.

Результаты версий сохраняются в `results/`. В этих же каталогах находятся итоговые маски, overlay-изображения и карты ошибок.

3 Описание алгоритма

3.1 Общая схема

1. подготовка исходного среза;
2. подавление шума без сильного размытия границ;
3. выделение ореха целиком;
4. выделение скорлупы;
5. выделение ядра;
6. выделение перегородок;
7. сборка итоговой карты классов.

Ниже подробное описание каждого шага.

3.2 Пошаговое описание

Шаг 1. Подготовка изображения

Сначала исходный TIFF-срез переводится в 8-битное полутоновое изображение. На этом этапе меняется только представление данных, а сама структура ореха остаётся той же. Результат этого шага показан на рисунке 1.

Рисунок 1 – исходный срез после перевода в 8-битное полутоновое изображение

Шаг 2. Сохраняющая границы фильтрация

Далее применяется bilateral filter с фиксированными параметрами

$$d = 7, \qquad \sigma_{\text{color}} = 40, \qquad \sigma_{\text{space}} = 9.$$

Здесь d задаёт размер окрестности фильтрации, σ_{color} отвечает за различие по яркости, а σ_{space} — за расстояние между пикселями. На практике это означает, что мелкий шум сглаживается, но резкие границы не размываются (рисунок 2).

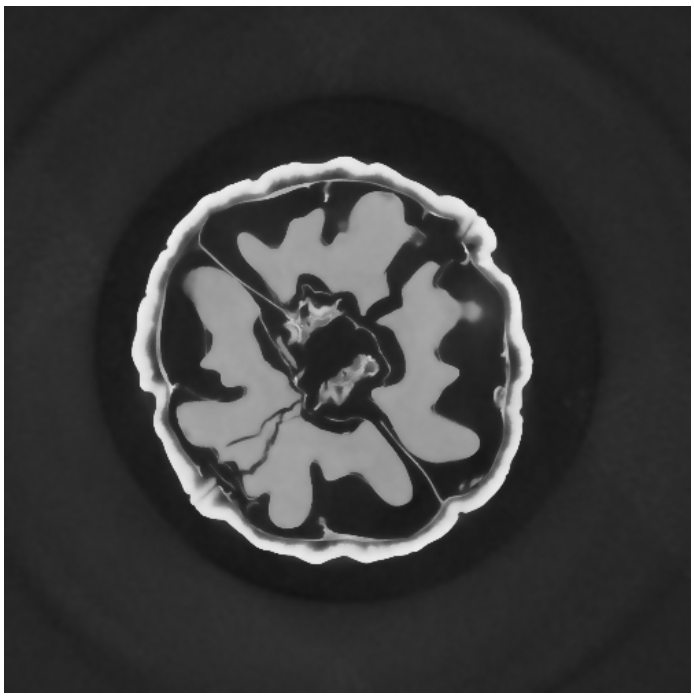


Рисунок 2 – результат bilateral filtering

Шаг 3. Выделение ореха целиком

После фильтрации строится бинарная маска всего ореха. Сначала вычисляется порог методом Оцу, затем берётся значение 0.72 от найденного порога. Пиксели выше этого уровня считаются принадлежащими объекту, остальные относятся к фону. После порогования выполняются морфологические операции закрытия и открытия, а затем оставляется самая большая связная компонента.

Размеры морфологических операций задаются через оценённый диаметр ореха d:

$$\texttt{nut_close} = \texttt{scaled}(d, 0.060, 11), \qquad \texttt{nut_open} = \texttt{scaled}(d, 0.015, 3).$$

Иначе говоря, чем больше сам орех на изображении, тем больше окно морфологической обработки. Результат этого шага — плотная маска, покрывающая весь орех без фона вокруг него (рисунок 3).

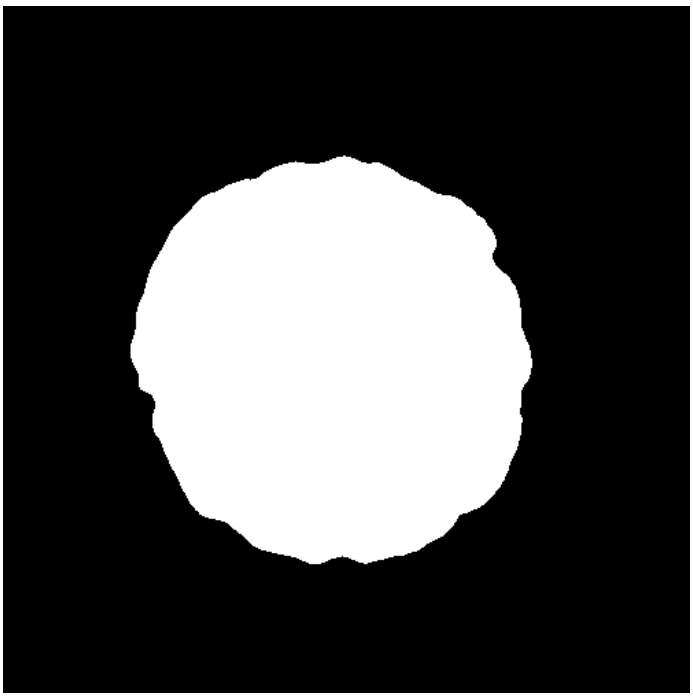


Рисунок 3 – маска ореха целиком

Шаг 4. Выделение скорлупы

После того как орех найден целиком, для каждого пикселя внутри ореха определяется расстояние до внешней границы. Затем берётся только узкая кольцевая область около края ореха. Именно в этой полосе и ищется скорлупа, поскольку она физически расположена у внешней границы.

Ширина полосы задаётся параметрами

$$\texttt{shell_lo} = \max(2, \mathrm{round}(0.006d)), \qquad \texttt{shell_hi} = \max(4, \mathrm{round}(0.048d)).$$

Здесь `shell_lo` задаёт внутреннюю границу полосы, а `shell_hi` — внешнюю. Далее внутри этой полосы выбирается порог по яркости: берётся более строгий из двух вариантов — порог Оцу для полосы или её средняя яркость. После этого выполняется закрытие, открытие и удаление очень маленьких компонент. На выходе получается тонкая яркая кольцевая маска скорлупы, пример которой приведён на рисунке 4.

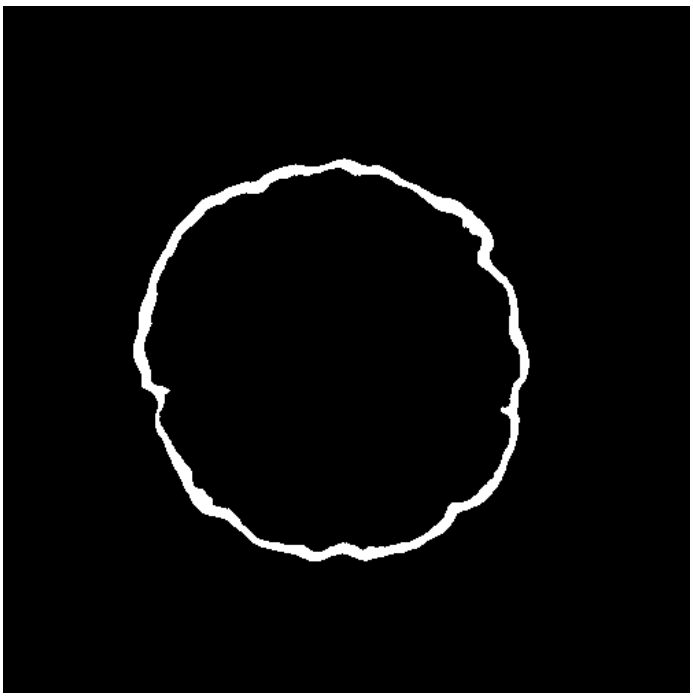


Рисунок 4 – выделенная маска скорлупы

Шаг 5. Выделение ядра

Из общей маски ореха убирается слегка расширенная маска скорлупы. Так получается внутренняя область, в которой может находиться ядро. Затем к изображению применяется адаптивная бинаризация с окном

$$\texttt{kernel_block} = \texttt{scaled}(d, 0.120, 21).$$

Решение о принадлежности пикселя к ядру принимается не по одному глобальному порогу на всё изображение, а по локальной яркости в окрестности. После этого выполняется открытие ядром 3×3, затем закрытие с ядром `kernel_close = scaled`(d,0.020,5), а в конце удаляются слишком маленькие компоненты.

версия v1 версия v2

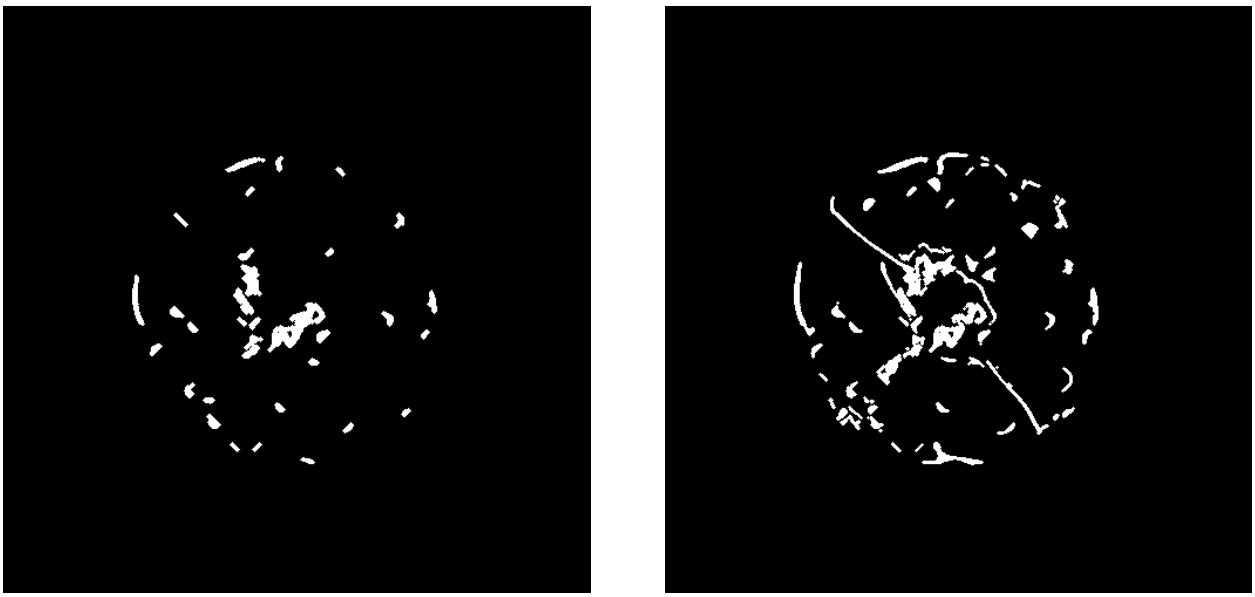
Рисунок 5 – выделенная маска ядра

Шаг 6. Выделение перегородок

Перегородки ищутся только во внутренней части ореха. Для этого используется направленный white top-hat. Эта операция усиливает тонкие светлые линии, которые по форме похожи на внутренние перегородки. Размер структурирующего элемента задаётся так:

$$\texttt{ridge_len} = \texttt{scaled}(d, 0.030, 7).$$

После этого кандидаты разделяются по яркости, затем выполняется закрытие ядром 3×3, дилатация ядром 3×3 и удаление слишком маленьких компонент. Итоговая маска перегородок для среза приведена на рисунке 6.

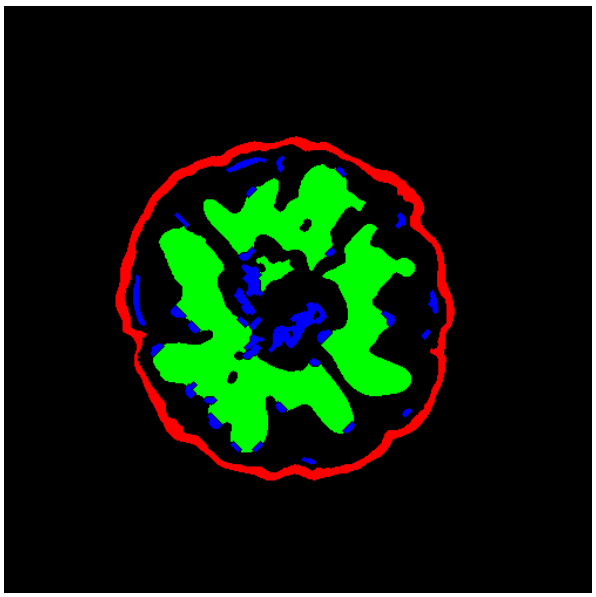


версия v1 версия v2

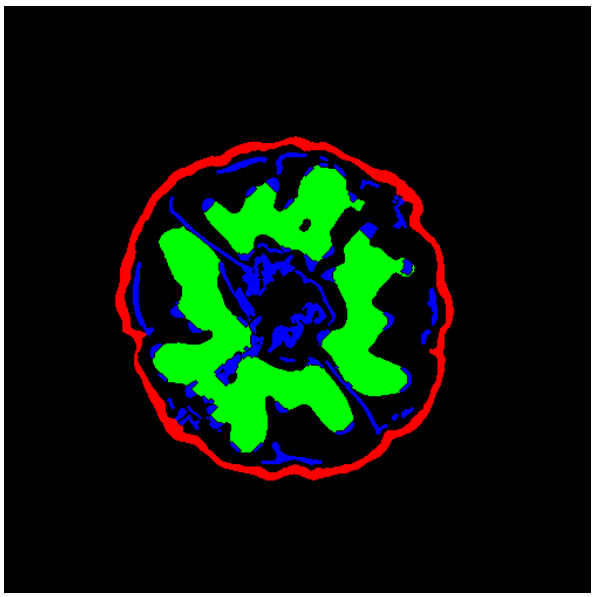
Рисунок 6 – выделенная маска перегородок

Шаг 7. Формирование итоговой карты классов

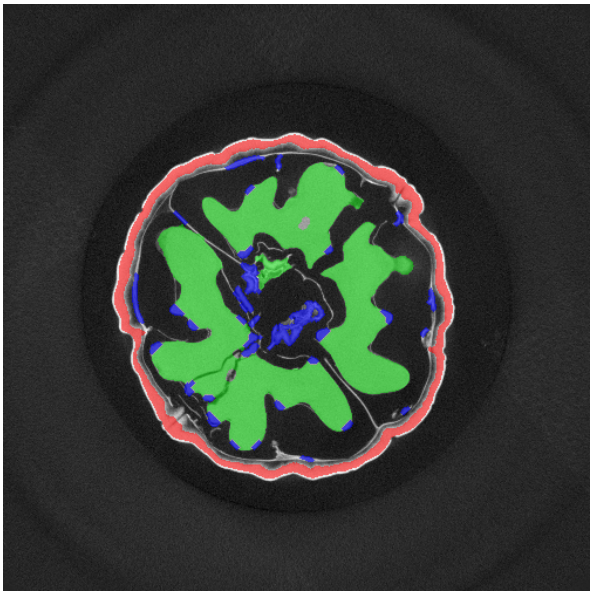
На последнем этапе все маски объединяются в одну карту классов. Пиксели фона получают метку 0, скорлупа — 1, ядро — 2, перегородки — 3. Для визуального контроля сохраняется цветная карта классов и наложение сегментации на исходный срез.



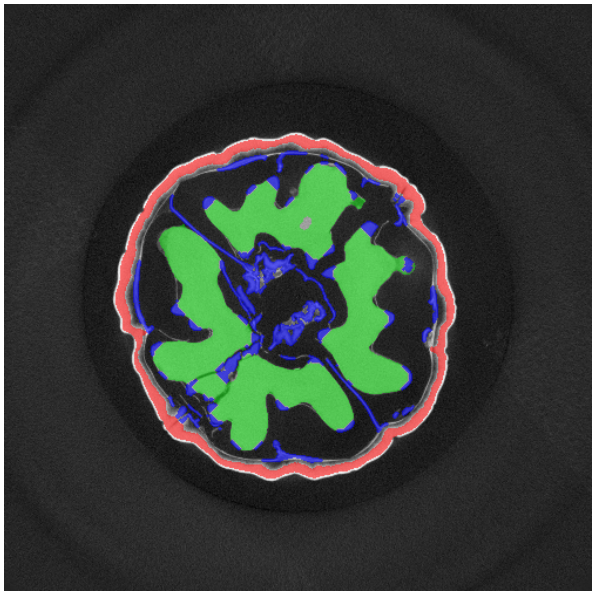
а — карта классов v1



б — карта классов v2



в — наложение на исходное изображение v1



г — наложение на исходное изображение v2

Рисунок 7 – итоговый результат

3.3 Изменения в версии v2

Во второй версии основная доработка касалась класса `septa`. В первой версии перегородки выделялись слишком грубо: тонкие внутренние структуры усиливались с помощью направленного `white top-hat`, после чего маска расширялась морфологическими операциями. Такой подход хорошо сохранял часть перегородок, но часто захватывал лишние области на внут-

ренних границах ядра.

Во второй версии для `septa` были добавлены дополнительные признаки: локальный контраст и модуль градиента. Кроме того, вместо простой пороговой маски была введена фильтрация компонент по площади, вытянутости, положению внутри ореха и перекрытию с полосой вокруг ядра. Это позволило лучше отличать настоящие перегородки от ложных ярких границ.

При этом итоговая версия `v2` не стала слишком жёсткой. По сравнению с промежуточным вариантом были ослаблены некоторые ограничения: зона поиска оставлена на расстоянии `shell_hi + 1` от скорлупы, порог по `ridge` сделан менее строгим, допуск по вытянутости уменьшен до 1.6, а ограничение на перекрытие с полосой вокруг ядра ослаблено до 0.98. После фильтрации компонент дополнительно выполняются небольшое замыкание и дилатация, чтобы восстановить разрывы в тонких перегородках. Это дало более сбалансированный результат.

4 Пользовательское описание

4.1 Режим сегментации

Программа запускается из командной строки. Для версии v1 используется команда вида

```
.\build\bin\Release\walnut_v1.exe --dataset ^
  .\dataset\Walnut2\Reconstructions ^
  --gt-dir .\gt_final --results .\results\v1
```

Для версии v2 используется аналогичная команда, но с исполняемым файлом `walnut_v2.exe` и выходным каталогом `results/v2`.

В результате работы программа сохраняет:

- исходное изображение;
- отфильтрованное изображение;
- маску ореха;
- маску скорлупы;
- маску ядра;
- маску перегородок;
- итоговую карту классов;
- overlay-изображение.

4.2 Режим оценки качества

Для расчёта метрик используется программа `walnut_metrics.exe`. Пример запуска:

```
.\build\bin\Release\walnut_metrics.exe ^
  --pred-root .\results\v1 --gt-root .\gt_final ^
  --csv .\results\v1\metrics.csv --tol 2
```

Для построения карт ошибок используется `walnut_error_map.exe`:

```
.\build\bin\Release\walnut_error_map.exe ^
```

```
--pred-root .\results\v2 --gt-root .\gt_final ^
--out-root .\results\v2\error_maps
```

Карты ошибок строятся отдельно для каждого класса. Зелёным цветом показаны правильно выделенные пиксели (TP), красным — ложноположительные пиксели (FP), синим — ложноотрицательные пиксели (FN). Фиолетовые области соответствуют пикселям, где произошла путаница между классами: для одного класса такой пиксель является FP, а для другого — FN.

5 Результаты

5.1 Критерии качества

Для оценки качества использовались усреднённые значения по классам. Основные метрики:

$$IoU = \frac{|P \cap G|}{|P \cup G|}, \qquad Dice = \frac{2|P \cap G|}{|P| + |G|},$$

где P — предсказанная маска класса, а G — ручная эталонная разметка. Также рассчитывались Precision, Recall и BoundaryF1.

5.2 Средние метрики версии v1

Усреднённые результаты версии `v1` по четырём срезам приведены в таблице 1.

Таблица 1 – Средние метрики качества для версии v1

Класс	IoU	Dice	BoundaryF1
shell	0.6791	0.8088	0.8953
kernel	0.8448	0.9158	0.7698
septa	0.1866	0.3137	0.4090

Из таблицы видно, что для классов `shell` и `kernel` алгоритм уже даёт устойчивый результат. Самым трудным классом остаются перегородки `septa`: они тонкие, местами слабоконтрастные и легко смешиваются с внутренними границами ядра. Поэтому именно для этого класса качество в версии `v1` остаётся самым низким.

5.3 Средние метрики версии v2

После доработки этапа выделения перегородок усреднённые результаты для версии `v2` приведены в таблице 2.

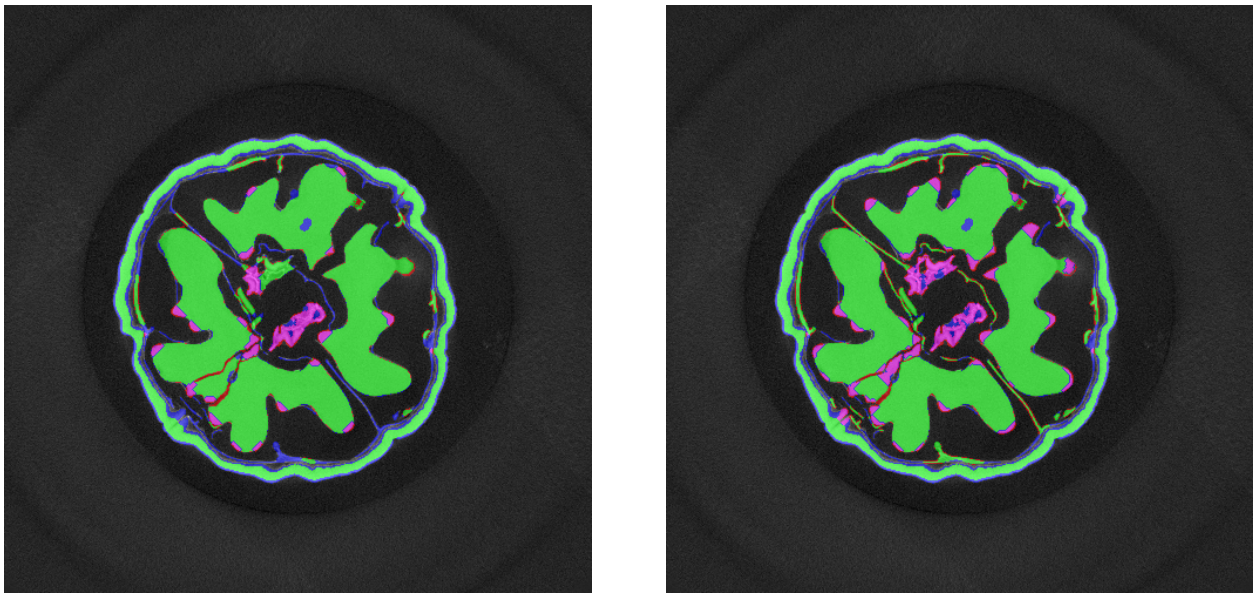
Таблица 2 – Средние метрики качества для версии v2

Класс	IoU	Dice	BoundaryF1
shell	0.6791	0.8088	0.8953
kernel	0.8389	0.9124	0.7396
septa	0.2865	0.4423	0.6436

По сравнению с версией `v1` в версии `v2` качество сегментации перегородок заметно выросло. Для класса `septa` значение `IoU` увеличилось с 0.1866 до 0.2865, `Dice` — с 0.3137 до 0.4423, а `BoundaryF1` — с 0.4090 до 0.6436. Метрики для `shell` изменились незначительно.

5.4 Анализ карт ошибок

На рисунке 8 показаны карты ошибок для версий `v1` и `v2`. Зелёным цветом показаны правильно выделенные пиксели, красным — ложноположительные, синим — ложноотрицательные. Фиолетовые участки соответствуют пикселям, где произошло пересечение между классами.



а — карта ошибок версии v1 б — карта ошибок версии v2

Рисунок 8 – Сравнение карт ошибок для representative-среза 193

По картам ошибок видно, что проблема версии `v1` связана с классом `septa`: алгоритм часто теряет части реальных перегородок и хуже попадает

в их границы. После доработки в версии `v2` совпадение по перегородкам стало лучше: правильные участки стали более непрерывными, а контуры перегородок лучше согласуются с эталонной разметкой. Это согласуется и с усреднёнными метриками, где для `septa` выросли `IoU`, `Dice` и `BoundaryF1`.

5.5 Проблемные случаи

Основной трудный класс в данной задаче — `septa`. Это связано с тем, что перегородки имеют малую толщину, местами слабый контраст и часто расположены рядом с резкими внутренними границами ядра. Из-за этого небольшое изменение порогов, морфологических операций или фильтрации компонент заметно влияет на итоговый результат.

В версии `v1` выделение перегородок было слишком грубым, поэтому качество по классу `septa` оставалось низким. В версии `v2` были добавлены дополнительные признаки и фильтрация компонент, а также ослаблены слишком жёсткие ограничения, из-за которых терялись истинные перегородки. Это позволило заметно улучшить выделение `septa`.

Таким образом, для классов `shell` и `kernel` качество уже можно считать хорошим, а основное улучшение между версиями `v1` и `v2` связано именно с классом `septa`.

6 Техническое описание

6.1 Установка

Перед сборкой нужно убедиться, что в корне проекта есть папка `gt_final` с размеченными срезами `fdk_pos1_<N>_labels_gt.tiff`. N - порядковый номер среза. Также, в корне проекта должна лежать папка с датасетом `dataset/Walnut2/Reconstructions`

Для сборки проекта требуется установленный OpenCV и CMake через `vcpkg`. Сборка выполняется из корня проекта следующими командами:

```
cmake -S . -B build
cmake --build build --config Release
```

6.2 Как воспроизвести результаты

Чтобы воспроизвести результаты, нужно выполнить следующие шаги из корня проекта:

1. Построение результата v1 для всех срезов:

```
.\build\bin\Release\walnut_v1.exe --dataset ^
  .\dataset\Walnut2\Reconstructions ^
  --gt-dir .\gt_final --results .\results\v1
```

2. Построение результата v2 для всех срезов:

```
.\build\bin\Release\walnut_v2.exe --dataset ^
  .\dataset\Walnut2\Reconstructions ^
  --gt-dir .\gt_final --results .\results\v2
```

3. Расчёт усреднённых метрик для v1:

```
.\build\bin\Release\walnut_metrics.exe ^
  --pred-root .\results\v1 --gt-root .\gt_final ^
  --csv .\results\v1\metrics.csv --tol 2
```

4. Расчёт усреднённых метрик для v2:

```
.\build\bin\Release\walnut_metrics.exe ^
  --pred-root .\results\v2 --gt-root .\gt_final ^
  --csv .\results\v2\metrics.csv --tol 2
```

5. Построение карт ошибок для v1:

```
.\build\bin\Release\walnut_error_map.exe ^
  --pred-root .\results\v1 --gt-root .\gt_final ^
  --out-root .\results\v1\error_maps
```

6. Построение карт ошибок для v2:

```
.\build\bin\Release\walnut_error_map.exe ^
  --pred-root .\results\v2 --gt-root .\gt_final ^
  --out-root .\results\v2\error_maps
```

7 Заключение

В работе реализован алгоритм сегментации КТ-срезов грецкого ореха на три класса: скорлупу, ядро и внутренние перегородки. Алгоритм построен на классических методах обработки изображений и реализован в виде консольного приложения на языке C++ с использованием библиотеки OpenCV.

В ходе работы были построены две версии алгоритма. Для класса `shell` в обеих версиях получены одинаковые устойчивые результаты. Для класса `kernel` качество также остаётся высоким. Наиболее трудным классом оказались внутренние перегородки `septa`, так как они имеют малую толщину, местами слабый контраст и часто расположены рядом с границами ядра.

Анализ карт ошибок и усреднённых метрик показал, что основное улучшение между версиями `v1` и `v2` связано именно с классом `septa`. После доработки алгоритма в версии `v2` для этого класса выросли все основные метрики качества: `IoU`, `Dice` и `BoundaryF1`. Это означает, что новая версия лучше выделяет реальные перегородки и точнее согласуется с ручной разметкой.

Таким образом, итоговая версия `v2` показала более качественный и более сбалансированный результат по сравнению с `v1`.

Список литературы

[1] OpenCV Documentation [Электронный ресурс]. URL: https://docs.opencv.org/4.x/.

[2] A dataset of X-ray microtomographies of walnuts [Электронный ресурс]. URL: https://zenodo.org/records/2686726.